

Football Intelligence Roadmap

Three pieces of plumbing are built and pushed. None of them has run against real data yet — that part needs your machine. Here is the order to do it in.

repo JBR995/Football-Statistics branch claude/football-intelligence-handover-xaxped base a88f806
commits 3 date 30 Aug 2026

WHERE IT STANDS

Built, but not yet fed

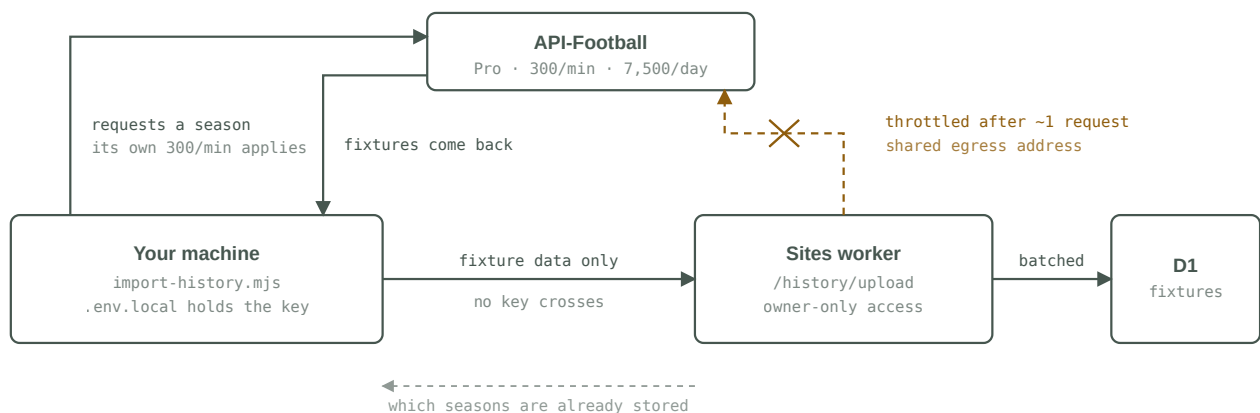
Every row below that says *ready* is code that works and has been exercised end to end against a local worker and a real D1 database — but with mock provider responses, because this environment’s egress policy blocks API-Football outright and cannot reach your Cloudflare database either. Nothing in the production numbers has moved since the handover.

PIECE	STATE	WHERE IT ACTUALLY IS
Historical seasons	UNCHANGED	11 of 60 competition-seasons stored. Two failed runs (League One 2022, League Two 2021) are still recorded as retryable.
Season upload	READY	Route plus local importer. Simulated to 60/60; a repeat run reported “60 already stored, 0 to import” without touching the provider.
Forecast snapshots	READY	Table, endpoints and the Models panel. Zero forecasts recorded — the record only fills going forward.
Statistics, line-ups, odds	READY	Three tables, a validated batch upload, a coverage report and a budgeted importer. Zero rows stored.
Deployment	BLOCKED ON YOU	Sites is still serving version 18 at a88f806. The Sites tooling only exists on your machine.

Why the importer runs on your machine

This is the constraint the whole ingestion design turns on. The API-Football subscription is healthy — Pro, active, and using about 66 of 7,500 daily calls. But the hosted worker leaves through a shared address whose per-minute allowance is spent after roughly one historical request, while the same key called from your machine showed 298 of 300 minute-requests still available.

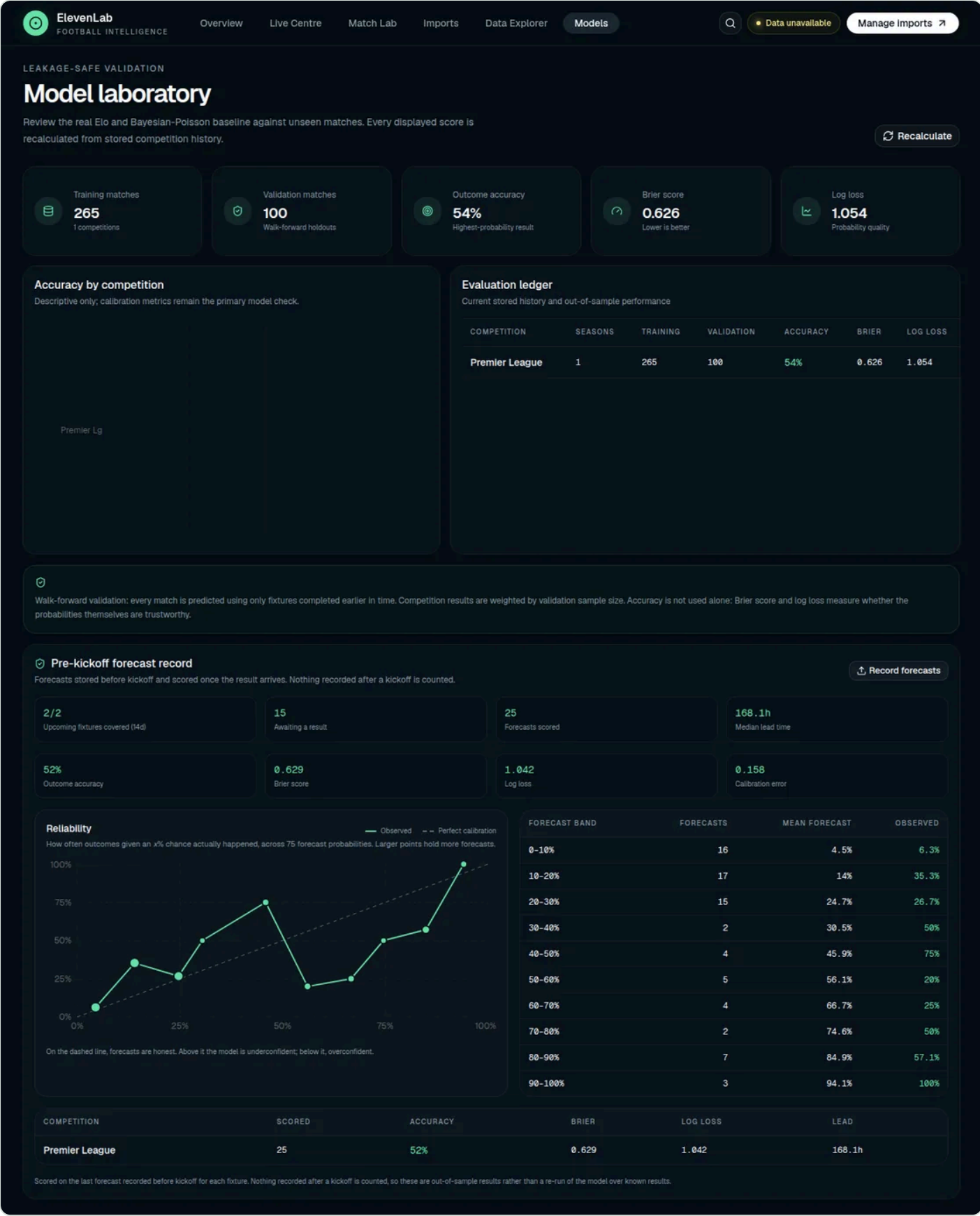
So the provider call moves to where the allowance is, and only the data crosses back. The key never leaves your machine.



The one hop that changes. The dashed amber path is the call that cannot complete from the worker. Everything else is the same pipeline, with the provider request relocated and the credential left behind. The importer reads the stored-season list first, which is what makes an interrupted run resumable.

What the build looks like now

These are screenshots of the branch running against a local database. The team names are placeholders and the numbers come from a simulated league — this environment cannot reach the provider or your production data, so a synthetic season was the only way to exercise the views. Read them for structure, not for results.



ElevenLab
FOOTBALL INTELLIGENCE

OverviewLive CentreMatch LabImportsData ExplorerModels

Data unavailableManage Imports ↗

LIVE DATA CONNECTION

Data explorer

Inspect exactly which provider fields are available and whether each data class is already stored or remains live-only. No demonstration metrics are shown.

Refresh

Live competition coverage

Current-season capabilities reported by the data provider.

Unavailable

The football data service is not connected.
API_FOOTBALL_KEY is not configured.

Historical storage readiness

What the research database can query today versus what remains available only in Live Centre. Counts are read from the database, not assumed.

Fixtures, results and scores

Stored

280 fixtures across 1 competition-seasons, 265 of them played.

Standings and team form

Calculated

Tables, home/away splits and last-six form are derived from stored results.

Baseline predictions

Calculated

Elo, Bayesian attack/defence and Poisson outputs use stored pre-kickoff evidence. Forecasts recorded before kickoff are scored in Models.

Shots, possession and passing

Stored

Shot, possession, passing and card counts for 265 of 265 matches, with expected goals where the provider supplies them.

Line-ups and formations

Stored

Starting elevens, substitutes, formation and coach for 265 of 265 matches. No model reads them yet.

Bookmaker odds

Stored

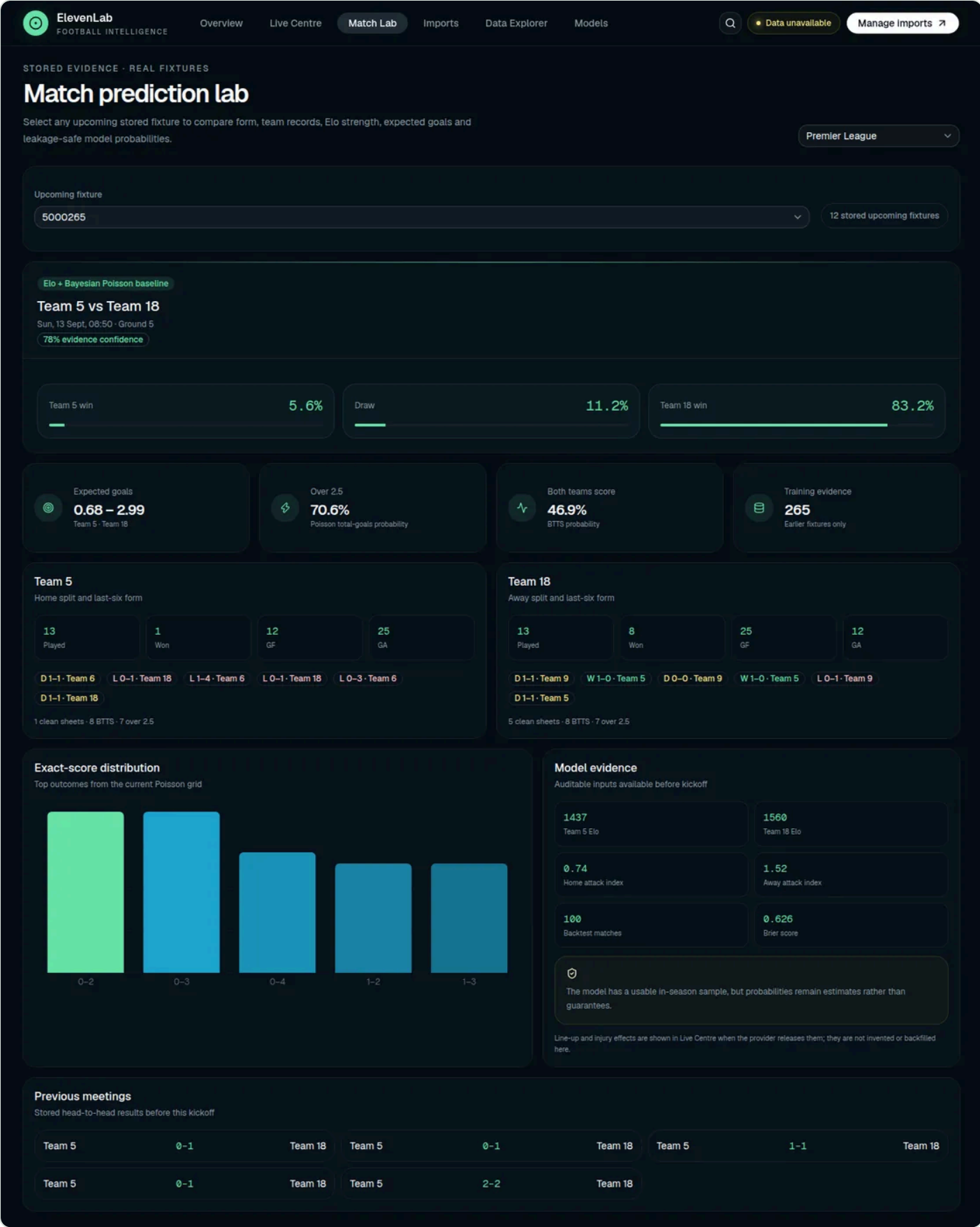
1X2, over/under 2.5 and both-teams-to-score prices for 280 of 280 matches, shown with the margin divided out.

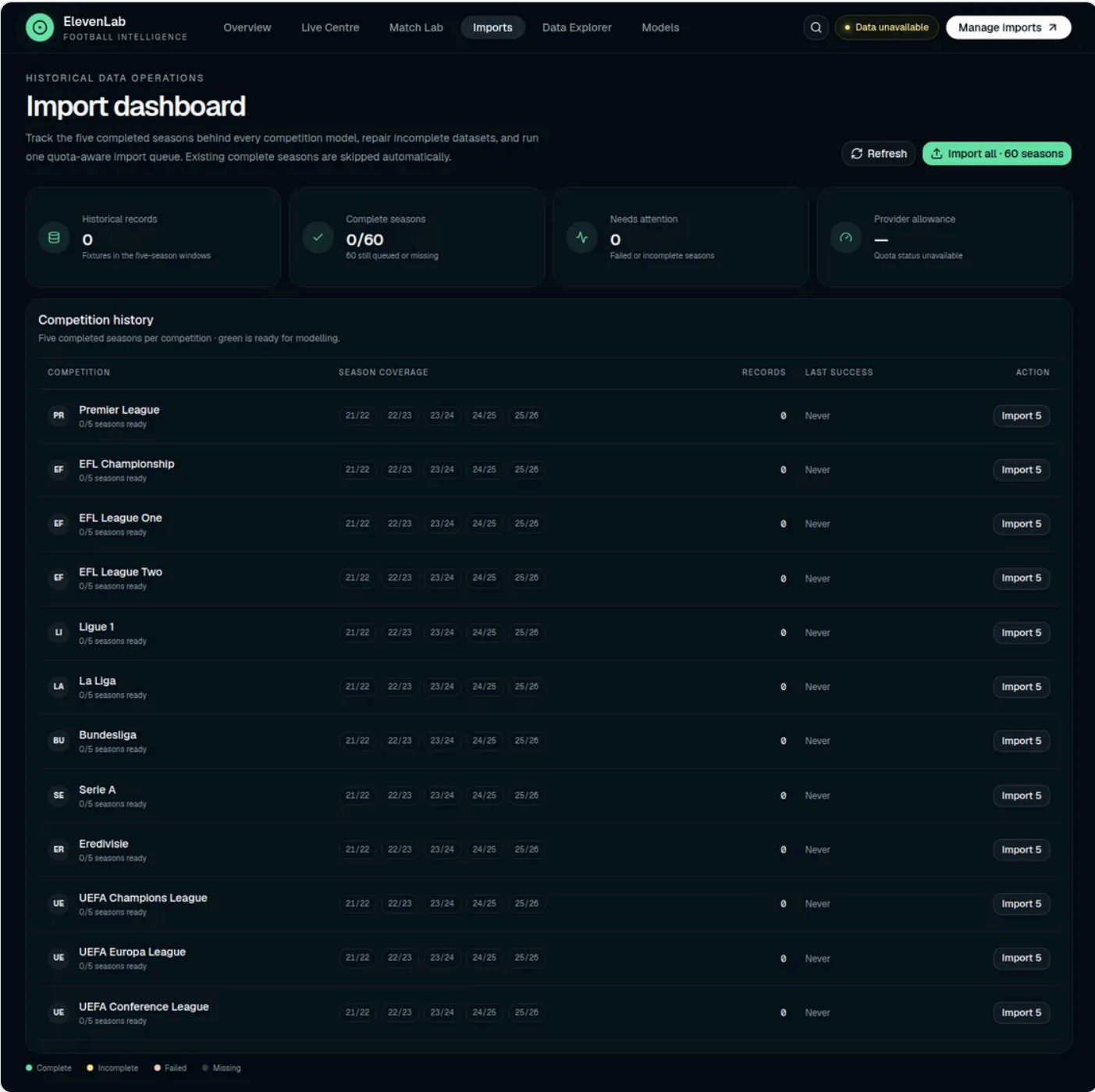
Injuries, suspensions and events

Live only

Not stored. Injuries are reported as a current state rather than a historical record, so a match-time snapshot has to be captured before kickoff to be usable.

Data explorer. The readiness panel now counts rows in the database instead of asserting a state, so it reports what is stored per class and says plainly when a class holds nothing. The empty provider panel above it is this environment having no API key – itself an honest render.





Imports. Reading 0/60 because the local test database holds only a 2026 season, outside the five-year window this dashboard tracks. Against production it currently reads 11/60; step two below is what changes that.

NEXT STEPS

In dependency order

The first three are a single evening’s work and they unlock everything after them. Steps four onward are the ones with real cost attached.

01 Deploy the branch

Nothing else can start: the importers post to routes that do not exist on version 18. Build, commit, push, package and save a Site version as the handover describes, then confirm the site is still owner-only.

```
pnpm build
```

 passes on this branch; the route list gained `/history/upload`, `/history/upload/match`, `/history/coverage` and `/predictions/snapshot`.

02 Run the season backfill to 60/60

Two to five minutes for the 49 outstanding seasons, including the two that failed on the provider's minute limit. Re-running is safe: completed seasons are skipped before any provider call is made.

```
node scripts/import-history.mjs \  
  --site https://elevenlab-football.choice-rice-1129.chatgpt.site \  
  --token <sites-bypass-token>
```

Then check Imports reads 60/60 and Models evaluates all twelve competitions rather than the three it can see today.

03 Start recording forecasts — and keep doing it

This is the step with a deadline attached, because a pre-kickoff record can only ever accumulate forward. Every week you do not run it is a week of evidence you cannot recover later.

```
curl -X POST .../api/football/predictions/snapshot \  
  -H 'content-type: application/json' -d '{"withinDays":7}'
```

Or press *Record forecasts* in Models. Do it once before each round of fixtures; re-running inside the same round stores nothing new unless the numbers actually moved.

NEXT, ONCE THE FIXTURES ARE IN

04 Backfill odds for two competitions first

Odds are the benchmark — a football model is judged against the closing price, not against a coin toss — and they are the cheapest of the three classes to justify. The Premier League and Championship across five seasons is roughly 4,700 fixtures, comfortably inside one day's allowance.

```
node scripts/import-match-detail.mjs --site ... --token ... \  
  --include odds --leagues 39,40 --budget 5000
```

05

Put the market next to the model

The match view already shows the market's 1X2 view with the bookmakers' margin divided out. The work is to carry that number into the prediction view and store it on each forecast snapshot, so the evaluation ledger can report the model's Brier score against the market's on the same fixtures.

That single column turns "is this any good?" from a judgement into a number, and it is the honest frame for everything after it.

06

Automate the two things that must not be forgotten

A scheduled task that refreshes results for finished fixtures and then snapshots the upcoming ones. Both are single HTTP calls now, so this is a Windows scheduled task or a Sites cron rather than a project.

THEN, ONCE THERE IS EVIDENCE TO COMPARE AGAINST

07

Backfill statistics, then fit on shots rather than goals

Expected goals and shot counts are a far lower-variance read on the same team than goals are — this is the largest single accuracy gain available, and it is why statistics are worth their call budget. Everything for all twelve competitions is roughly 21,500 fixtures per class; nine days at the daily cap for all three classes, so target it rather than sweeping it.

08

Dixon-Coles, and a real comparison

Add the low-score correction and exponential time decay the current Poisson baseline lacks, then score it against the baseline and against the market on Brier and log loss — not on accuracy, which is misleading at these margins. Only add gradient boosting if it beats a well-specified Dixon-Coles on those two numbers.

09

Calibration by competition and season

The reliability machinery exists; splitting it by competition and season, with confidence bands on thin samples, is what makes it usable for deciding where the model can be trusted.

10

The conversational layer, last

It should read the stored evidence and the recorded forecasts. Building it before those are reliable would just give a fluent voice to numbers nobody has checked.

Three decisions I left open

Keep the upload route?

The handover treated it as temporary, to be deleted after the migration. But the egress throttle is not going away, and the same path serves every future re-import.

I would keep it. It is validated and guarded like the existing POST rather than being a hole. Say the word and it goes.

How much match detail?

All three classes across all twelve competitions is roughly 64,000 provider calls — about nine days of the daily allowance, spent on data no model reads yet.

Odds and statistics for the Premier League and Championship first. Skip line-ups entirely until something consumes them.

Line-ups as JSON blobs

Each team's starting eleven and bench are stored as JSON on one row, rather than normalised into a players table.

Fine while nothing queries across players. The day a model wants "matches without their first-choice keeper", it needs normalising first.

What I did not build, and why

- **Injuries.** The provider publishes them as a current state, not a historical record. Storing them usefully means capturing a snapshot before each kickoff, which belongs with the automation in step six rather than with a backfill.
- **Match events.** Goal and card timelines are a fourth call per fixture, and no planned model reads them. Cheap to add to the same importer when one does.
- **Model comparison.** Deliberately left until the odds are stored — comparing two models to each other, with no benchmark either has to beat, is how you end up confident in the wrong one.

- **Anything against real data.** This environment's egress policy blocks API-Football, football-data.co.uk and understat.com, and cannot reach your D1. Every number in the screenshots is from a simulated league, and every claim about production comes from the handover rather than from a check I was able to make.

Three commits on `claude/football-intelligence-handover-xaxped`, based on `a88f806`.

5427672 local-machine ingestion for historical seasons

651470c pre-kickoff forecast snapshots and scoring

435ceab match statistics, line-ups and odds storage